Machine Translated by Google

[Chap 12] Artificial Neural Network Implementation and the KERAS Package

Machine Translated by Google

Installation of Keras packages

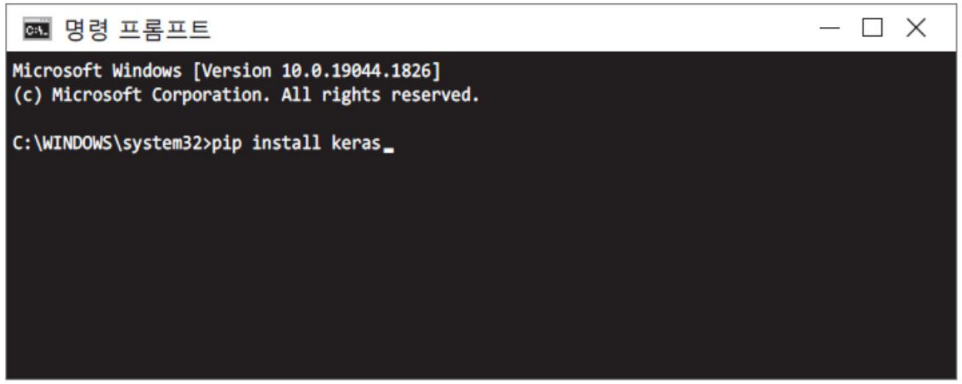
- Keras – A

  package for implementing artificial neural networks and deep

  learning – Originally a separate project by Google, but now distributed alongside TensorFlow

- Installing Keras – pip

  install keras (Mac: pip3 install keras)



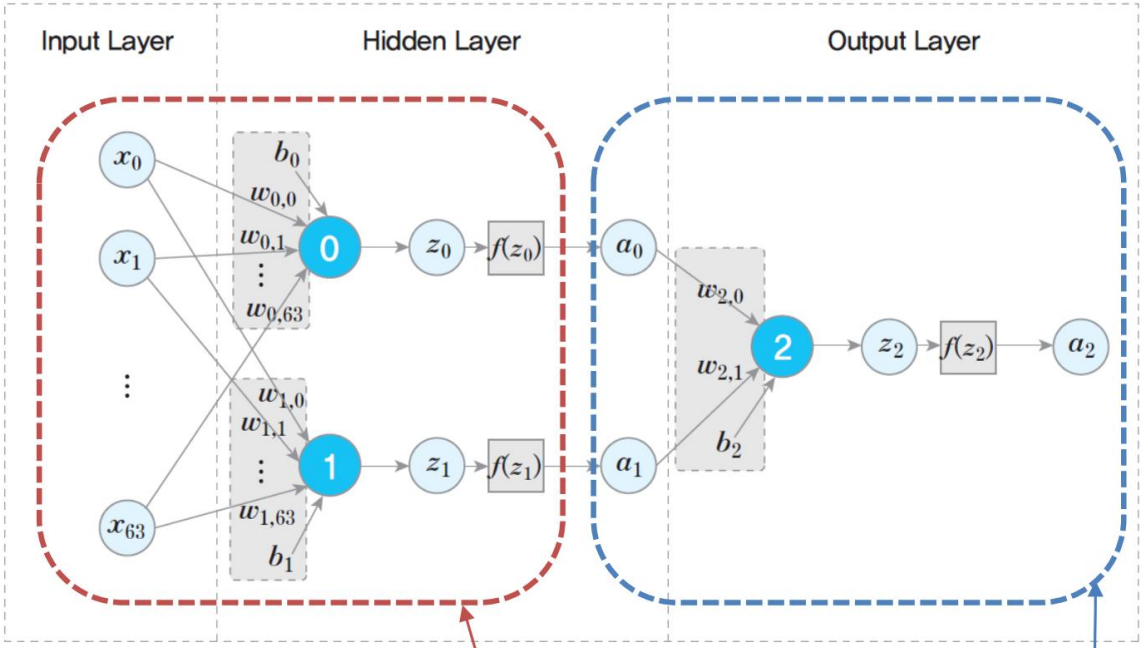
[그림 12-1] pip를 이용한 Keras 설치

Machine Translated by Google

Implementation of an artificial neural network to distinguish between odd and even numbers in handwriting

- Artificial Neural Network Structure

  (Same as Chapter 11)



[그림 12-2] Keras로 구현하려는 인공신경망 구조

- Implement the above neural network in Keras

```
model = keras.Sequential([
    keras.layers.Dense(2, activation='sigmoid', input_shape=(64,)),
    keras.layers.Dense(1, activation='sigmoid')
])
```

Machine Translated by Google

Artificial Neural Network for Distinguishing Odd/Even Handwriting (Continued)

• Keras Modeling Explanation

```
1  model = keras.Sequential([
2      keras.layers.Dense(2, activation='sigmoid', input_shape=(64,)),
3      keras.layers.Dense(1, activation='sigmoid')
4  ])
```

– Write each layer of the neural network sequentially as a list inside keras.Sequential()

• Although it consists of multiple lines, there is a list inside keras.Sequential(), and in this list two Form with values – (Row

2) Hidden layer modeling •

Indicate that there are 2 neurons in the hidden

layer • Input layer handled by the input_shape
option – Set to have 64 values in the input layer

• Dense ensures that every node is connected to the previous layer • Set the

activation function to sigmoid – (Row 3) Output Layer

Modeling • Consists of 1 neuron

• Activation function is

sigmoid • Connected to all
previous layers

Machine Translated by Google

Artificial Neural Network for Distinguishing Odd/Even Handwriting (Continued)

- Loss function and optimization engine settings

```
model.compile(optimizer='SGD', loss='binary_crossentropy')
```

– Configure loss function and optimizer within model.compile() – loss option: Set loss function

- Set the loss function to the entropy function binary_crossentropy. • The types of loss functions and usage criteria will be explained later.

– optimizer option: optimization engine settings

- The optimization engine is set to SGD (Stochastic Gradient Descent).

- Model Training –

Input x_train and output y_train, and set the number of iteration epochs to 2000.

```
model.fit(x_train, y_train, epochs=2000)
```

Machine Translated by Google

Artificial Neural Network for Distinguishing Odd/Even Handwriting (Continued)

- Full code and execution results –

  Metrics such as precision and recall have reached 94% – Compared to the implementation results of the previous chapter

  Fast execution speed and high performance

  - Differences in optimization algorithms (explained later)

```
Epoch 1/2000
45/45 [==============================] - 0s 488us/step - loss: 0.6999
Epoch 2/2000
45/45 [==============================] - 0s 421us/step - loss: 0.6924
Epoch 3/2000
45/45 [==============================] - 0s 421us/step - loss: 0.6869
...
Epoch 2000/2000
45/45 [==============================] - 0s 443us/step - loss: 0.1443
[[168  11]
 [ 10 171]]
              precision    recall  f1-score   support

         0.0       0.94      0.94      0.94       179
         1.0       0.94      0.94      0.94       181

    accuracy                           0.94       360
   macro avg       0.94      0.94      0.94       360
weighted avg       0.94      0.94      0.94       360
```

예제 ex12-1.py

```python
import numpy as np
from tensorflow import keras
from sklearn import datasets

dataset = datasets.load_digits()
x_data = (dataset.data/15.0).astype(np.float32)
y_data = (dataset.target %2).astype(np.float32)

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data,
  test_size=0.2)

model = keras.Sequential([
    keras.layers.Dense(2, activation='sigmoid', input_shape=(64,)),
    keras.layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='SGD', loss='binary_crossentropy')
model.fit(x_train, y_train, epochs=2000)

predictions = model.predict(x_test)
predLabels=[]
for y in predictions:
    if y<0.5: predLabels.append(0)
    else: predLabels.append(1)

from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

print(confusion_matrix(y_test, predLabels))
print(classification_report(y_test, predLabels))
```

Machine Translated by Google

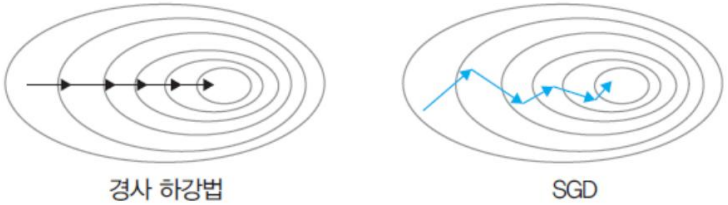
SGD (Stochastic Gradient Descent) Algorithm

- Problems with pure Gradient Descent (GD): Becomes very slow as n increases – calculations must be performed on all n training data points to find the gradient.

$$\frac{\partial L}{\partial w_{31}} = \frac{1}{n}\sum_{i=1}^{n}(a_3(i) - y(i))a_1(i)$$

$$\frac{\partial L}{\partial w_{11}} = \frac{1}{n}\sum_{i=1}^{n}(a_3(i) - y(i))w_{31}(a_1(i)(1 - a_1(i)))x_1(i)$$

- SGD (Stochastic Gradient Descent): Stochastic Gradient Descent – Iteratively samples a single data point, calculates the gradient, and modifies the weights – While it cannot accurately calculate the direction of the gradient, it converges overall in a downward direction over a long period.



[그림 12-3] 순수한 경사하강법(왼쪽)과 SGD(오른쪽)의 비교

- Mini Batch SGD –

  Calculates gradients by randomly sampling a fixed number of data points instead of a single record

  - Batch: The amount of data used to calculate weights at once •
  Keras is set to 32 by default

Machine Translated by Google

Batch and epoch

- Epoch: Going through the entire data once
- Batch: The number of data points used for a single gradient calculation

- Relationship between batches and epochs in SGD
  - Pure GD: Uses all data for a single gradient calculation, so it takes 1 epoch
    Modify weights once (1 epoch = 1 batch)
  - Pure SGD: Since the gradient is calculated from a single data point, the weights are modified as many times as the number of data points (n) per epoch (1 epoch = n batches)
  - Mini-batch SGD: Since gradients are calculated using data of batch size (m), weights are modified n/m times per epoch (1 epoch = n/m batch)

Machine Translated by Google

Optimization algorithms supported by Keras

nonlinear optimization (비선형 최적화 optimization)

〈표 12-1〉 Keras의 최적화 알고리즘 종류

최적화 알고리즘	설명
SGD	'Stochastic Gradient Descent'. 확률적 경사하강법. 그래디언트에 학습률을 곱한 만큼 파라미터를 수정.
Adagrad	'ADAptive GRADient algorithm'. 자주 수정된 파라미터는 학습률을 낮추고 덜 자주 수정된 파라미터에 학습률을 높이는 식으로 파라미터별로 학습률을 달리하여 수렴성을 개선. 학습률을 각 파라미터별 기존 그래디언트의 제곱합의 제곱근으로 나눔. 학습이 진행됨에 따라 학습률이 0에 가까워지는 단점이 있음
RMSprop	'Root Mean Square PROPagation'. Adagrad에서 학습률이 0에 가까워지는 되는 문제를 해결하기 위해 파라미터별 그래디언트의 제곱합 대신 지수평활법에 의한 가중이동평균의 제곱근으로 학습률을 나눔.
Adadelta	Adagrad에서 학습률이 0에 가까워지는 되는 문제를 해결하기 위해 전체 기간의 그래디언트 수정 정도 대신 최근 일정기간 동안의 그래디언트 수정 정도만 고려하도록 함
Adam	'ADAptive Moment estimation'의 약어로서, 최근 그래디언트의 수정 정도뿐 아니라 그래디언트의 모멘텀(관성력)을 고려하여 수렴성 개선과 더불어 부분 최적에 빠지는 문제를 개선
Adamax	Adam에서 학습률 조정을 위한 계수에 L^{∞}(infinity norm), 즉 벡터의 최대값을 사용하도록 변형. 임베딩(embedding) 등 일부 응용에서 Adam보다 우수한 성능을 보임.
Nadam	Adam에 NAG(Nesterov Accelerated Gradient)를 적용한 방법. 여기서 NAG는 모멘텀을 반영할 때 파라미터의 새로운 값 지점의 그래디언트를 함께 고려하여 수렴 안정성을 향상시키는 기법임.

Machine Translated by Google

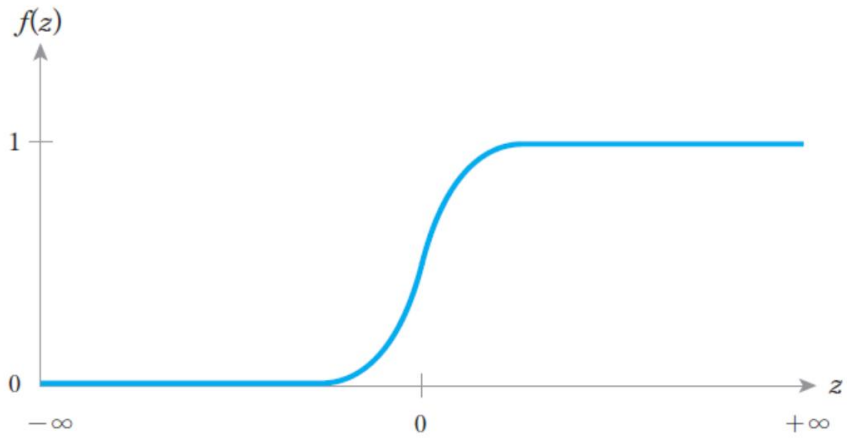
Types of activation functions

- Sigmoid function (activation='sigmoid') – (ÿÿ, ÿ)

  ÿ (0, 1) – Used in

  binary classification

$$f(z) = \frac{1}{1+e^{-z}}$$



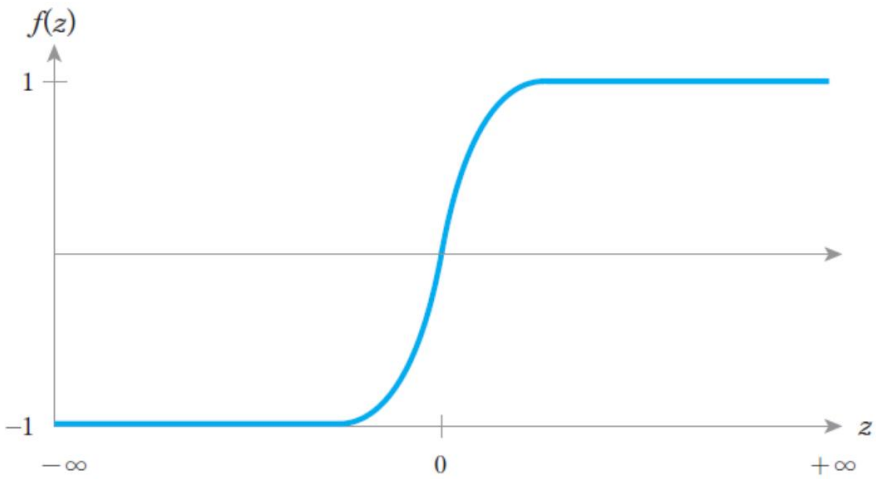
[그림 12–4] 시그모이드 활성화 함수

Machine Translated by Google

Types of Activation Functions (continued)

- Hyperbolic tangent function (activation='tanh') – (ÿÿ, ÿ) **ÿ** (-1, 1) – Used in binary classification
  – Similar in purpose to sigmoid, but tanh converges slightly better during actual training

$$f(z) = \tanh(z) = \frac{1-e^{-2z}}{1+e^{-2z}}$$



[그림 12-5] 하이퍼볼릭 탄젠트 활성화 함수

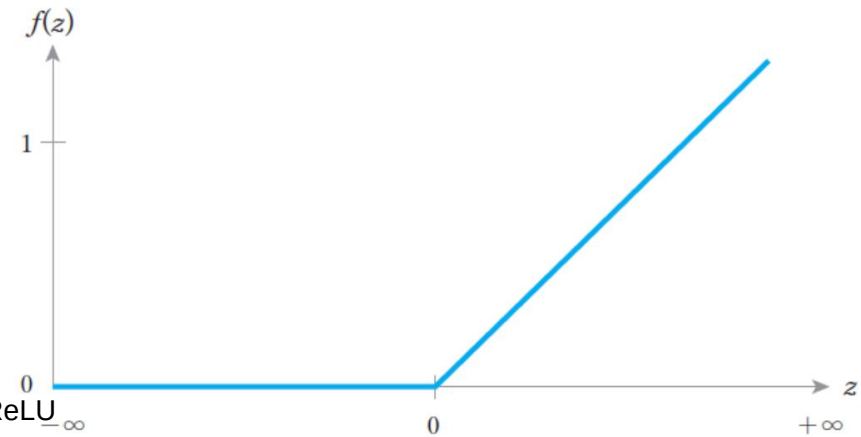
Machine Translated by Google

Types of Activation Functions (continued)

- Gradient vanishing problem
  - A phenomenon where learning does not work well because the gradient decreases sharply and becomes nearly zero when z becomes a small or large value.
  - Common problem of sigmoid function and hyperbolic tangent function

- ReLU (Rectified Linear Unit) function (activation='relu')
  - Solving the vanishing gradient problem – If z is positive, pass it as is; if negative, convert it to 0
    - (ÿÿ, ÿ) **ÿ** (0, ÿ)
  - Advantages • Solves the vanishing gradient problem by preserving gradients in positive intervals • Fast computation speed due to its linear function form – Deep learning hidden layers almost always use ReLU

$$f(z) = \max(0, z)$$



[그림 12-6] ReLU 활성화 함수

Machine Translated by Google

Types of Activation Functions (continued)

- Softmax function
  - Used for multi-class classification • The output layer has multiple neurons • The weighted sum of each neuron

When saying 1, 2, 3,

$$f(z_1) = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$f(z_2) = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$f(z_3) = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

Machine Translated by Google

Softmax Function: Dog, Cat, Teddy Bear Classification Problem

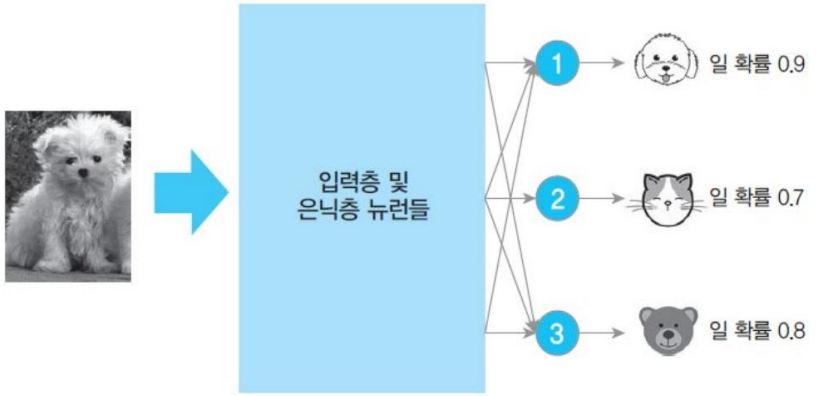
- In photo 1, when applying the $_1$ $_2$ = 0.85, $_3$ = 1.39 ramen, sigmoid function = 2.2

$$f(z_1)=\frac{1}{1+e^{-2.2}}=0.9,\quad f(z_2)=\frac{1}{1+e^{-0.85}}=0.7,\quad f(z_3)=\frac{1}{1+e^{-1.39}}=0.8$$

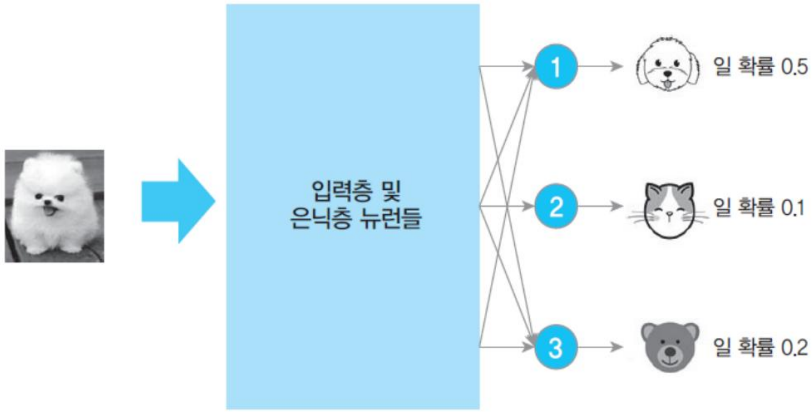
- In photo 2, when applying the sigmoid function, $_1$ $_2$ $_3$ = ÿ1.372 ramen, = 0, = ÿ2.19

$$f(z_1)=\frac{1}{1+e^{0}}=0.5,\quad f(z_2)=\frac{1}{1+e^{2.19}}=0.1,\quad f(z_3)=\frac{1}{1+e^{1.37}}=0.2$$

- Problems with the simple application of sigmoid
  - In photo number 1, the puppy and the teddy bear are being confused, but
    In number 2, the probability of it being a puppy is much higher than in the others.
  - However, the probability value itself indicates a higher probability for puppy number 1.
  - A judgment derived solely from the output value of each neuron
    Calculating the degree of certainty is problematic, and how the
    probability of a relevant judgment compares to the probability of other judgments
    Probability calculations reflecting whether there is a difference are needed.



[그림 12-7] 강아지 사진 판별 예 1



[그림 12-8] 강아지 사진 판별 예 2

Machine Translated by Google

Softmax function (continued)

- Comparison of the softmax function results for Photo 1 and Photo 2

$$f(z_1) = \frac{e^{z_1}}{e^{z_1}+e^{z_2}+e^{z_3}} = \frac{e^{2.2}}{e^{2.2}+e^{0.85}+e^{1.39}} = 0.59$$

$$f(z_2) = \frac{e^{z_2}}{e^{z_1}+e^{z_2}+e^{z_3}} = \frac{e^{0.85}}{e^{2.2}+e^{0.85}+e^{1.39}} = 0.15$$

$$f(z_3) = \frac{e^{z_3}}{e^{z_1}+e^{z_2}+e^{z_3}} = \frac{e^{1.39}}{e^{2.2}+e^{0.85}+e^{1.39}} = 0.26$$

$$f(z_1) = \frac{e^{z_1}}{e^{z_1}+e^{z_2}+e^{z_3}} = \frac{e^{0}}{e^{0}+e^{-2.19}+e^{-1.37}} = 0.73$$

$$f(z_2) = \frac{e^{z_2}}{e^{z_1}+e^{z_2}+e^{z_3}} = \frac{e^{-2.19}}{e^{0}+e^{-2.19}+e^{-1.37}} = 0.08$$

$$f(z_3) = \frac{e^{z_3}}{e^{z_1}+e^{z_2}+e^{z_3}} = \frac{e^{-1.37}}{e^{0}+e^{-2.19}+e^{-1.37}} = 0.19$$

  – Confirmed that the probability of a puppy was higher in the second case

- The Softmax function combines the output values of multiple neurons to reflect the difference in relative confidence – the degree of confidence reflecting the difference in relative probabilities while making the sum of the total probabilities equal to 1

  Advantage of calculating

  – Commonly used as an activation function for multi-classification problems with two or more neurons in the output layer

Machine Translated by Google

loss function

- mean_square_error: mean square error

$$Loss = \frac{1}{n}\sum_{i=1}^{n}(y_i - a_i)^2$$

- binary_crossentropy: binary cross-entropy
  - Used in binary classification

$$Loss = -\frac{1}{n}\sum_{i=1}^{n}\{y_i \log(a_i) + (1-y_i)\log(1-a_i)\}$$

- categorical_crossentropy: Categorical cross-entropy – Used in multi-classification (3 or more classification classes)

$$Loss = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{c} y_{ij}\log(a_{ij})$$

  - Example) 3-Class Classification. Assuming the following for 2 data points : • Correct answers: (1,0,0), (1,0,0) • Predicted values: (0.59, 0.15, 0.26), (0.73, 0.08, 0.19) • Categorical cross-entropy is calculated as follows

$$\begin{aligned} Loss &= -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{c} y_{ij}\log(a_{ij}) = -\frac{1}{2}\sum_{i=1}^{2}\sum_{j=1}^{3} y_{ij}\log(a_{ij}) \\ &= -\frac{1}{2}[\{1 \cdot \log(0.59) + 0 \cdot \log(0.15) + 0 \cdot \log(0.26)\} \\ &\quad + \{1 \cdot \log(0.73) + 0 \cdot \log(0.08) + 0 \cdot \log(0.19)\}] \\ &= 0.4212 \end{aligned}$$

Machine Translated by Google

Loss function (continued)

- sparse_categorical_crossentropy : Sparse categorical cross-entropy

  – The formula is identical to categorical cross-entropy, but it is distinguished and used based on the representation method of the ground truth data type.

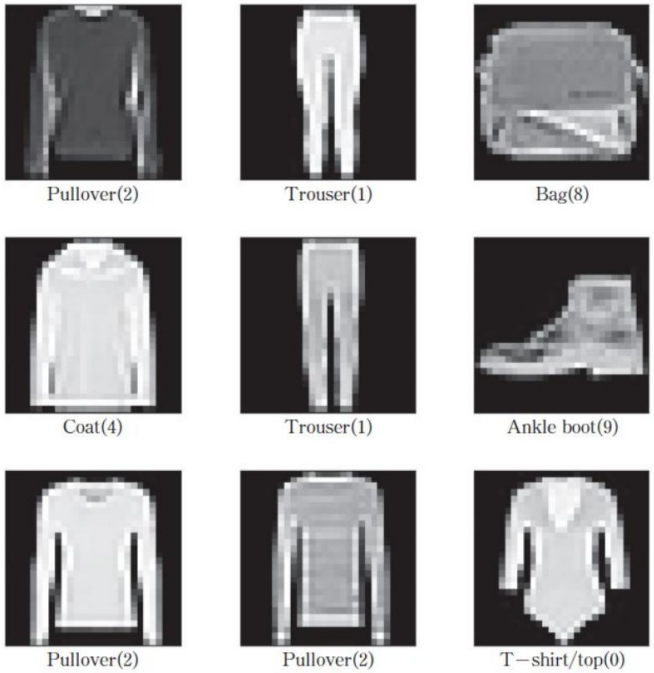
  – The correct answer values are (1, 0, 0), (0, 1, 0), (0, 0, 1), and each correct answer value has as many elements as the number of classes If branches are represented as vectors, use categorical cross-entropy. • Problems with high memory usage for representing correct answers.

  – Use sparse_categorical_crossentropy if the answer value is expressed as an integer number like 0, 1, or 2.
    - Memory saving possible

Machine Translated by Google

Image Recognition Example: Fashion MNIST

- Each data point consists of a black-and-white photograph of clothing and a number indicating which of the 10 categories the clothing belongs to – each image is 28×28 pixels, and each pixel has a brightness value between 0 and 255
- 60,000 training data points and 10,000 test data points



[그림 12-9] Fashion MNIST의 이미지 샘플

〈표 12-2〉 Fashion MNIST 데이터셋의 정답값 의미

정답 라벨	의미
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Machine Translated by Google

Examining the data

- Read Fashion MNIST data and output the data format and the first image

예제 ex12-2.py

```
1  from tensorflow import keras
2  fashion_mnist = keras.datasets.fashion_mnist
3  (train_images, train_labels), (test_images, test_labels) = \
4      fashion_mnist.load_data()
5  print(train_images.shape)
6  print(train_labels.shape)
7  print(test_images.shape)
8  print(test_labels.shape)
9
10 print('Train label 0 = ', train_labels[0])
11 print(train_images[0])
12
13 import matplotlib.pyplot as plt
14 plt.imshow(train_images[0])
15 plt.colorbar()
16 plt.show()
```

(Lines 2–4) Read data

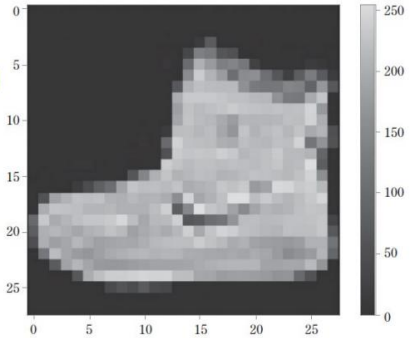
(Lines 5–8) Output training and test data format

```
(60000, 28, 28)
(60000,)
(10000, 28, 28)
(10000,)
```

(Lines 10–11) Output data value 0

```
Train label 0 =  9
[[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]  ...
```

(Lines 13–16) Output image of data 0



[그림 12-10] 학습용 데이터 0번의 이미지

Machine Translated by Google

Image recognition artificial neural network implementation

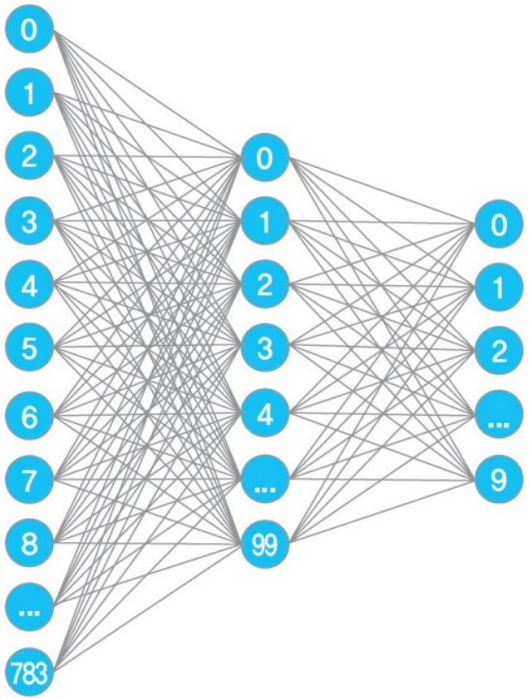
• Neural Network Structure Design –

Input Layer: Most simply, 784 (= 28×28) pixel values • Divide input values by 255 to convert them into values between 0 and 1 – Hidden Layer: The key is determining the number of layers and the number of neurons in each layer, but here, it is a single layer with neurons Set the quantity to 100

```
train_images = train_images/255.0
test_images = test_images/255.0
```

– Output layer: 10 neurons are needed, corresponding to the number of classes to be distinguished.

• Each neuron determines whether the input image belongs to the corresponding label.



[그림 12-11] Keras로 구현하려는 인공신경망 구조

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(100, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
```

Machine Translated by Google

Implementation of an Artificial Neural Network for Image Recognition (Continued)

- Setting up the optimization algorithm and loss function

```
model.compile(optimizer = 'adam', loss = 'sparse_categorical_crossentropy')
```

- Provide learning images and answers to proceed with learning

```
model.fit(train_images, train_labels, epochs=1)
```

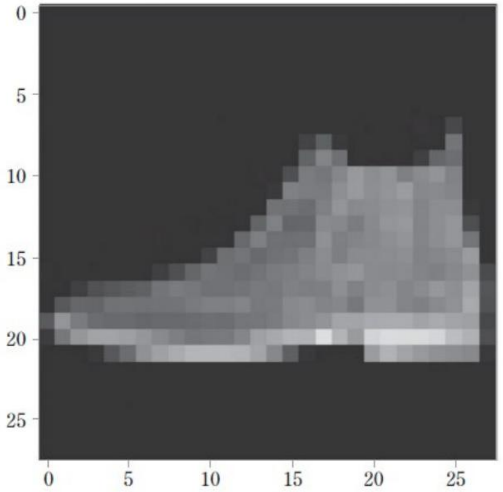
Prediction by applying test data

```
predictions = model.predict(test_images)
```

- Output the first test image and predicted values

```
import matplotlib.pyplot as plt
print('Test label 0 =', test_labels[0])
print('Prediction 0 =', predictions[0])
plt.imshow(test_images[0])
plt.show()
```

```
Test label 0 = 9
Prediction 0 = [2.6515195e-06 1.5080832e-07 3.8006117e-06 1.7151444e-06 2.2985461e-05
 3.2011930e-02 4.0852105e-06 3.1448203e-01 3.7465435e-03 6.4972407e-01]
```



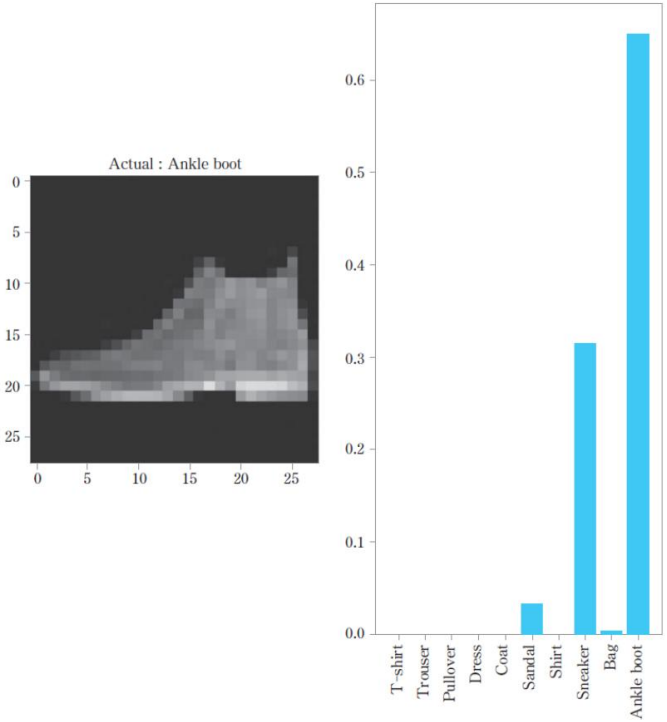
[그림 12-12] 테스트 이미지 0번

Machine Translated by Google

Implementation of an Artificial Neural Network for Image Recognition (Continued)

- Output each neuron's predicted value as a bar graph

```
import matplotlib.pyplot as plt
names = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat',
         'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
for i in range(30):
    print('Actual : ', test_labels[i], names[test_labels[i]])
    plt.subplot(1,2,1)
    plt.imshow(test_images[i])
    plt.title('Actual : '+names[test_labels[i]])
    plt.subplot(1,2,2)
    plt.bar(names, predictions[i])
    plt.xticks(rotation='vertical')
    plt.show()
```



[그림 12-13] 테스트 이미지 0번에 대한 예측

Machine Translated by Google

Performance evaluation

- Currently, the 'predictions' variable contains 10 probability values for each data point.
  - To use the number with the highest probability as the prediction, find the index of the highest value for each vector in `predictions` using `np.argmax` and store this value as the prediction.

```
import numpy as np
predicted_labels = []
for p in predictions:
    predicted_labels.append(np.argmax(p))
```

- Output prediction performance indicators

```
from sklearn.metrics import classification_report
print(classification_report(test_labels, predicted_labels))
```

```
              precision    recall  f1-score   support

           0       0.88      0.78      0.82      1000
           1       0.99      0.96      0.98      1000
           2       0.73      0.81      0.77      1000
           3       0.83      0.93      0.88      1000
           4       0.87      0.63      0.73      1000
           5       0.98      0.96      0.97      1000
           6       0.63      0.76      0.69      1000
           7       0.93      0.97      0.95      1000
           8       0.98      0.96      0.97      1000
           9       0.96      0.95      0.96      1000

    accuracy                           0.87     10000
   macro avg       0.88      0.87      0.87     10000
weighted avg       0.88      0.87      0.87     10000
```